# hblibCodeoutput User Manual
# for hblibCodeoutput.sty v20261108T0900

Haluk Bingol

Contents


**1. Getting started** 1
- 1.1. The idea: code and output stay in files 1
- 1.2. Real outputs: run the programs, keep what they print 2

**2. Quick reference** 2

**3. Building blocks** 2
- 3.1. \lstcode: code from a file 2
  - 3.1.1. Part of a file: firstline, lastline, firstnumber 3
- 3.2. \lstoutputand \lstterminal 3

**4. Code and output together** 3
- 4.1. \lstcodedown: output below 3
- 4.2. \lstcoderight: output on the right 4
- 4.3. The step macros 4

**5. In a slide deck** 5
- 5.1. Output on the second slide 5
- 5.2. \lstcodeoverlaystepand \slidegrid 5

**6. Typed and inline code** 5
- 6.1. Short code typed in the .tex 5
- 6.2. Inline code 5

**7. Characters and colors** 6
- 7.1. Turkish letters and currency signs 6
- 7.2. Colors 6

**8. Notes and troubleshooting** 6
- 8.1. A missing file 6
- 8.2. Troubleshooting 7


# 1. Getting started

## 1.1. The idea: code and output stay in files

```latex
\documentclass[11pt]{amsart}
\usepackage{../../cis000_latex/lib/hblibArticle}       % an article; or hblibPresentation in a deck
\usepackage{../../cis000_latex/lib/hblibCodeoutput}    % after it
...
\lstcoderight{codeoutput/Grades.java}{Java}{codeoutput/Grades.txt}
```

A lecture folder:

```
lecture/
  cis-array1D.tex
  run_examples.sh
  codeoutput/
    Grades.java     the program
    Grades.in       keyboard input
    Grades.txt      what it printed
```

- Every macro takes relative paths, relative to the .tex file; nothing is copied into the .tex.
- Load hblibCodeoutput after hblibArticle (or hblibPresentation); it loads hblibCommon from its own folder.
- Optional: needed only for code and output *from files* in boxes (it loads tcolorbox and textpos).

- The same macros work in article and amsart (lab descriptions, exams) and in beamer (Sec. 5).

### 1.2. Real outputs: run the programs, keep what they print

Outputs are never typed by hand. A script runs every program and writes its `.txt`:

```
#!/bin/bash
# run_examples.sh: compile and run every program in codeoutput/, keep its output
cd codeoutput
for f in *.java; do
    name="${f%.java}"
    javac "$f"
    if [ -f "$name.in" ]; then java "$name" < "$name.in" > "$name.txt"
    else                       java "$name"              > "$name.txt"; fi
done
```

- Change a program, run `sh run_examples.sh`, compile the document: code and output always match.
- A missing file gives a red box, not an error (Sec. 8.1), so a document compiles before its outputs exist.

## 2. Quick reference

Optional arguments are in `[brackets]`; paths are relative to the .tex file.

| Write | Shows |
|---|---|
| `\lstcode[code opts]{code}{Language}` | code, Sec. 3.1 |
| `\lstoutput[output opts]{output}` | output, Sec. 3.2 |
| `\lstterminal[opts]{transcript}` | a terminal session, Sec. 3.2 |
| `\lstcodedown[code opts][output opts]{code}{Language}{output}` | output below, Sec. 4.1 |
| `\lstcoderight[width][code opts][output opts]{code}{Language}{output}` | output right, Sec. 4.2 |
| `\lstcodedownstep[code opts][output opts]{code}{Language}{output}` | below; in a deck on slide 2, Sec. 4.3 |
| `\lstcoderightstep[width][code opts][output opts]{code}{Language}{output}` | right; in a deck on slide 2, Sec. 4.3 |
| `\lstcodeoverlaystep[code opts][output opts][width]{x,y}{code}{Language}{output}` | on top, slide 2 (decks), Sec. 5.2 |
| `\slidegrid` | a 1 cm grid (decks), Sec. 5.2 |
| *from hblibCommon (loaded by hblibCodeoutput):* | |
| `\hCodeJ{}`, `\code{}`, `\hCodeB{}`, `\hCode{}` | inline code, Sec. 6.2 |
| `\hbLira`, `\hbRupee` | ₺, ₹, Sec. 7.1 |
| styles `codestyle`, `outputstyle`, `terminalstyle`, `codesnippet`, `texsnippet` | `style=...`, Sec. 6.1 |

## 3. Building blocks

### 3.1. `\lstcode`: code from a file

```latex
\lstcode[options]{code path}{Language}
\lstcode{Grades.java}{Java}
```

Grades.java — Java

```java
public class Grades {
    public static void main(String[] args) {
        int[] marks = {72, 85, 90, 64, 78};
        int sum = 0;
        for (int m : marks) {
            sum += m;
        }
        double average = (double) sum / marks.length;
        System.out.println("count   = " + marks.length);
        System.out.println("sum     = " + sum);
        System.out.println("average = " + average);
    }
}
```

- Title bar: the path, and the language on the right; line numbers on the left.
- `Language` is a listings language: `Java`, `C`, `Python`, `bash`, ...
- `options` are listings options, e.g. `basicstyle=\ttfamily\scriptsize`, `numbers=none`.

### 3.1.1. Part of a file: `firstline`, `lastline`, `firstnumber`

latex

```latex
\lstcode[firstline=3, lastline=7]{Grades.java}{Java}                  % numbered 1 ... 5
\lstcode[firstline=3, lastline=7, firstnumber=3]{Grades.java}{Java}   % numbered 3 ... 7, as in the file
```

Grades.java — Java

```java
        int[] marks = {72, 85, 90, 64, 78};
        int sum = 0;
        for (int m : marks) {
            sum += m;
        }
```

Grades.java — Java

```java
        int[] marks = {72, 85, 90, 64, 78};
        int sum = 0;
        for (int m : marks) {
            sum += m;
        }
```

With `firstnumber`, "line 5" in the text is line 5 of the file. Other options: `linerange={1-2,9-11}`, `numbers=none`.

## 3.2. `\lstoutput` and `\lstterminal`

latex

```latex
\lstoutput{Grades.txt}       % output only
\lstterminal{Session.txt}    % a terminal session
```

Output — Grades.txt

```
count   = 5
sum     = 389
average = 77.8
```

Terminal — Session.txt

```
$ javac Hello.java
$ ls
Hello.class
Hello.java
$ java Hello
Merhaba, Ayşe!
Price: 25 ₺
```

- `\lstoutput`: a dark box, like a terminal; "Output" and the path in the title bar.
- `\lstterminal`: lines starting with `$` are the commands; they are highlighted.

# 4. Code and output together

## 4.1. `\lstcodedown`: output below

latex

```latex
\lstcodedown[code options][output options]{code path}{Language}{output path}
\lstcodedown[firstline=8, lastline=11, firstnumber=8]{Grades.java}{Java}{Grades.txt}
```

Grades.java — Java

```
        double average = (double) sum / marks.length;
        System.out.println("count   = " + marks.length);
        System.out.println("sum     = " + sum);
        System.out.println("average = " + average);
```

Output — Grades.txt

```
count   = 5
sum     = 389
average = 77.8
```

## 4.2. \lstcoderight: output on the right

```
\lstcoderight[code width][code options][output options]{code path}{Language}{output path}
\lstcoderight[0.64][basicstyle=\ttfamily\scriptsize]{Grades.java}{Java}{Grades.txt}
```

latex

Grades.java — Java

```
public class Grades {
    public static void main(String[] args) {
        int[] marks = {72, 85, 90, 64, 78};
        int sum = 0;
        for (int m : marks) {
            sum += m;
        }
        double average = (double) sum / marks.length;
        System.out.println("count   = " + marks.length);
        System.out.println("sum     = " + sum);
        System.out.println("average = " + average);
    }
}
```

Output — Grades.txt

```
count   = 5
sum     = 389
average = 77.8
```

code width: a fraction of the line width (default 0.56); the output gets the rest. In an article the two parts are top-aligned minipages; in a deck they are beamer columns.

## 4.3. The step macros

\lstcodedownstep and \lstcoderightstep take the same arguments as \lstcodedown and \lstcoderight. In a deck the output appears on the second slide (Sec. 5); in an article there are no slides, so they show the output at once, exactly like the macros without step. A deck's frames can therefore be copied into lecture notes unchanged.

```
\lstcodedownstep[basicstyle=\ttfamily\scriptsize, firstline=3, lastline=8, firstnumber=3]{Grades.java}{Java}{
    Grades.txt}
\hQuestion{What is the average?}
```

latex

Grades.java — Java

```
        int[] marks = {72, 85, 90, 64, 78};
        int sum = 0;
        for (int m : marks) {
            sum += m;
        }
        double average = (double) sum / marks.length;
```

Output — Grades.txt

```
count   = 5
sum     = 389
average = 77.8
```

**Q.** What is the average?

# 5. In a slide deck

Load hblibPresentation instead of hblibArticle. Every frame that uses these macros must be `[fragile]`.

## 5.1. Output on the second slide

```latex
\documentclass[aspectratio=169,10pt,compress]{beamer}
\usepackage{../../hblib/hblibPresentation}
\usepackage{../../hblib/hblibCodeoutput}
...
\begin{frame}[t,fragile] \frametitle{Average}
  \lstcoderightstep[0.66][basicstyle=\ttfamily\scriptsize]{codeoutput/Grades.java}{Java}{codeoutput/Grades.txt}
  \small \hQuestion{What is the average?}  \uncover<2->{$389 / 5 = 77.8$}
\end{frame}
```

- Slide 1: the code. Slide 2: the output too.
- The output's space is kept on slide 1, so nothing jumps; the rest of the frame is not delayed.
- `\lstcodedownstep`: the same, with the output below.

## 5.2. `\lstcodeoverlaystep` and `\slidegrid`

When the code fills the frame, the output floats on top of it on slide 2.

```latex
\lstcodeoverlaystep[code options][output options][output width]{x,y}{code path}{Language}{output path}
\lstcodeoverlaystep[basicstyle=\ttfamily\scriptsize][][5cm]{9cm,3.6cm}{Grades.java}{Java}{Grades.txt}
```

- `x,y`: the output's upper-left corner, from the slide's upper-left corner (x right, y down).
- Output width: default `0.45\textwidth`.

To find `x` and `y`, put a labeled 1 cm grid over the slide and read the corner from it, e.g. `{9cm,3.6cm}`. It works for any slide size ($16 \times 9$ cm for 16:9, $12.8 \times 9.6$ cm for 4:3).

```latex
\begin{frame}[t,fragile]
  \slidegrid        % a labeled 1 cm grid over the slide; remove it when done
  ...
```

**Note.** Both place things at fixed positions on the *page*, so they are meant for slides. In an article use `\lstcodedown` or `\lstcoderight` instead.

# 6. Typed and inline code

These come from hblibCommon: they work without hblibCodeoutput too.

## 6.1. Short code typed in the .tex

```latex
\begin{lstlisting}[style=codesnippet, language=Java]
double[] t = {14.0, 16.5};   // one name
System.out.println(t[1]);    // index 1
\end{lstlisting}

\begin{lstlisting}[style=terminalstyle,
  backgroundcolor=\color{termbg},
  basicstyle=\ttfamily\footnotesize\color{termfg}]
$ java Hello
Merhaba, Ayşe!
\end{lstlisting}
```

```java
double[] t = {14.0, 16.5};   // one name
System.out.println(t[1]);    // index 1
```

```
$ java Hello
Merhaba, Ayşe!
```

Styles: `codesnippet` (framed, light), `codestyle` (numbered, no frame), `outputstyle`, `terminalstyle`, `texsnippet` (LaTeX source, as in the boxes of this manual).

## 6.2. Inline code

```latex
\hCodeJ{int n = a.length;}   \hCodeB{ls -l *.java}   \code{for (int m : marks)}   \hCode{int n = a.length;}
```

`int n = a.length;` `ls -l *.java` `for (int m : marks)` `int n = a.length;`

| Command | Gives | Where |
|---|---|---|
| \hCodeJ, \code | Java, syntax colored, typed as is | running text (in a deck: a [fragile] frame) |
| \hCodeB | bash, typed as is | running text (in a deck: a [fragile] frame) |
| \hCode | plain, colored; escape _ \{ \} | everywhere, also titles and captions |

\hCodeJ and \hCodeB cannot be used in section titles, captions or footnotes, and their text must not contain % or #.

# 7. Characters and colors

## 7.1. Turkish letters and currency signs

```latex
\lstcoderight[0.62][basicstyle=\ttfamily\scriptsize]{Hello.java}{Java}{Hello.txt}
In text: 25\,\hbLira{} and 25\,\hbRupee{}
```

Hello.java — Java

```java
public class Hello {
    public static void main(String[] args) {
        String name = "Ayşe";
        System.out.println("Merhaba, " + name + "!");
        System.out.println("Price: 25 ₺");
    }
}
```

Output — Hello.txt

```
Merhaba, Ayşe!
Price: 25 ₺
```

In text: 25 ₺ and 25 ₹

- Code and output print Turkish and Latin-1 letters, € £ ¥ °, quotes, dashes and no-break spaces.
- ₺ and ₹ are drawn (few TeX fonts have them): \hbLira, \hbRupee.
- Wrongly decoded text (“mojibake”, e.g. AÄŸaÃ§ for Ağaç) also prints, so it can be shown.

## 7.2. Colors

| Name | RGB | Used for |
|---|---|---|
| codeframe | 40, 90, 160 | code box frame, title bar |
| codebg | 248, 248, 248 | code box background |
| codekw | 0, 70, 160 | keywords |
| codestr | 30, 130, 60 | strings |
| codecmt | 120, 120, 120 | comments, line numbers |
| termbg | 35, 38, 45 | output box background |
| termframe | 70, 75, 85 | output box frame |
| termfg | 225, 230, 235 | output text |
| termcmd | 255, 214, 102 | terminal commands |

The colors are defined in hblibCommon. Change one with \definecolor: in the preamble for the whole document, inside a group {...} for that part only.

```latex
{\definecolor{codeframe}{RGB}{150,40,40}
 \lstcode[firstline=3, lastline=4, firstnumber=3]{Grades.java}{Java}}
```

Grades.java — Java

```java
        int[] marks = {72, 85, 90, 64, 78};
        int sum = 0;
```

# 8. Notes and troubleshooting

## 8.1. A missing file

```latex
\lstcoderight{codeoutput/NotYet.java}{Java}{codeoutput/NotYet.txt}
```

```
missing file:  codeoutput/NotYet.java
```

```
missing file:  codeoutput/NotYet.txt
```

A red box names the missing file; the document still compiles. Create the file, or check the path (relative to the .tex file, letters in the right case).

## 8.2. Troubleshooting

| Message or problem | Cause | Fix |
|---|---|---|
| Undefined control sequence \tcbuselibrary | a library older than v20261104T2350 | update hblibCodeoutput.sty |
| errors inside a frame with code (decks) | frame not `fragile` | `[t,fragile]` |
| red box “missing file” | wrong path, or no output yet | fix the path; run the programs |
| the output is shown on slide 1 (decks) | a one-slide macro | use a `*step` macro |
| line numbers start at 1 with `firstline` | listings counts printed lines | add `firstnumber=` |
| the output title runs into the margin | a long path in a narrow column | a wider output (smaller code width) |
| `\hCodeJ` fails in a title or caption | listings inside an argument | `\hCode` |
| an output box at an odd place on the page (articles) | `\lstcodeoverlaystep` places by page position | `\lstcodedown` or `\lstcoderight` |
| File ‘. . . /hblibCommon.sty’ not found | hblibCommon missing | all hb libraries in one folder |
| Command \hCodeJ already defined | old and new libraries mixed | update all hb libraries together |
| Undefined control sequence \beamer@sectionintoc | .toc/.aux left from the beamer version | delete the .aux, .toc, .out files, compile again |